

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

CENTRO REGIONAL DE CHIRIQUÍ

**FACULTAD DE INGENIERÍA EN SISTEMAS
COMPUTACIONALES**

**LICENCIATURA EN INGENIERÍA EN SISTEMAS Y
COMPUTACIÓN**

Curso: Herramientas de programación IV

Asignación: Investigación I

Estudiante: David Luo 4-825-618

Docente: Napoleón Ibarra

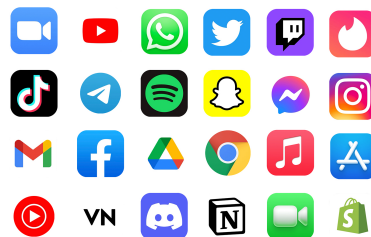
Fecha de entrega: 27/03/2026

Primer semestre 2026

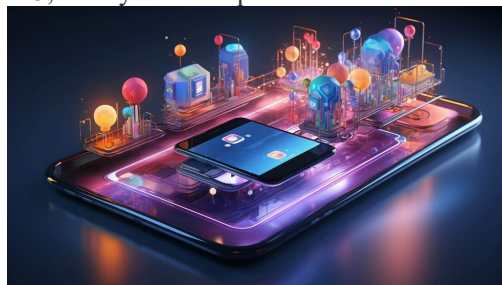
DESARROLLO:

1. **Glosario ilustrado:** Apps, Aplicaciones Web para móviles, Aplicaciones nativas, Aplicaciones híbridas, IDE, SDK, SDK Manager, Carpeta Java, Carpeta Res, Carpeta Layout, Carpeta Values, Activity, Fragments, Conten Provider, Broadcast Receiver, Services, XML, Android Manifest, Emulador, Dispositivo, Android, OS, Android Studio, APK, Bug, Conten Provider, Swap, UI, Widget.

1. **Apps:** Abreviatura de "Applications". Software diseñado para ejecutar un grupo de funciones, tareas o actividades coordinadas para beneficio del usuario en dispositivos informáticos.



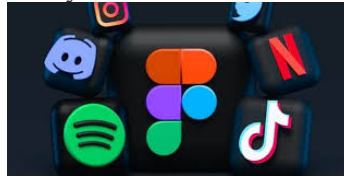
2. **Aplicaciones Web para móviles:** Aplicaciones que no se instalan en el dispositivo, sino que se ejecutan en un servidor remoto y se visualizan a través del navegador del móvil utilizando tecnologías como HTML5, CSS y JavaScript.



3. **Aplicaciones nativas:** Programas desarrollados en el lenguaje específico del sistema operativo (Java/Kotlin para Android, Swift para iOS), lo que permite un acceso total al hardware y el mejor rendimiento posible.



4. **Aplicaciones híbridas:** Aplicaciones que combinan elementos nativos y web. Se desarrollan con lenguajes web (multiplataforma) y se encapsulan en un contenedor nativo que permite su distribución en tiendas como la Play Store.



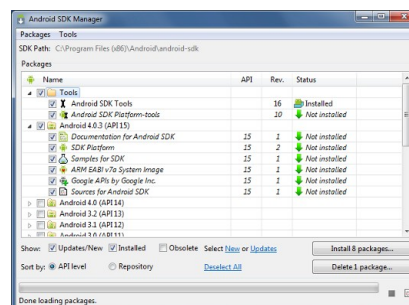
5. **IDE (Integrated Development Environment):** Entorno de Desarrollo Integrado. Aplicación informática que proporciona servicios integrales para facilitar al desarrollador la escritura, depuración y compilación de código fuente.



6. **SDK (Software Development Kit):** Kit de Desarrollo de Software. Conjunto de herramientas, bibliotecas, código de muestra y documentación que permite a los programadores crear aplicaciones para una plataforma específica.



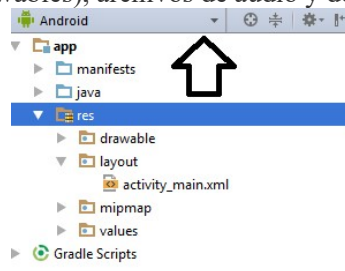
7. **SDK Manager:** Herramienta integrada en los IDEs (como Android Studio) que permite gestionar, instalar y actualizar las diferentes versiones de las APIs de Android y las herramientas de construcción necesarias.



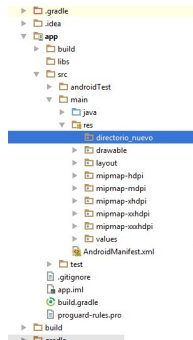
8. **Carpeta Java**: Directorio dentro de un proyecto de Android que contiene los archivos de código fuente escritos en Java o Kotlin, organizados por paquetes.



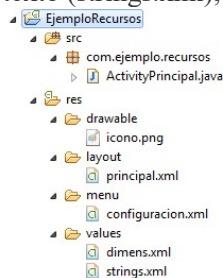
9. **Carpeta Res**: Abreviatura de "Resources". Directorio que almacena todos los recursos que no son código, como imágenes (drawables), archivos de audio y definiciones de valores.



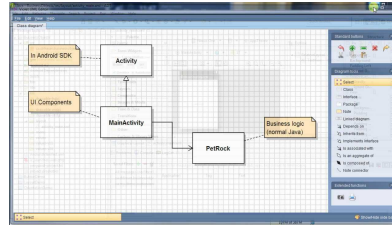
10. **Carpeta Layout**: Subdirectorio de la carpeta res que contiene los archivos XML que definen la estructura visual y el diseño de las interfaces de usuario de la aplicación.



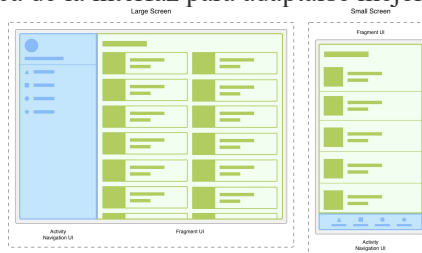
11. **Carpeta Values**: Directorio que contiene archivos XML para definir constantes reutilizables en toda la aplicación, como cadenas de texto (strings.xml), colores (colors.xml) y dimensiones.



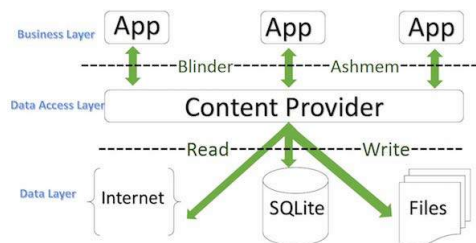
12. **Activity**: Componente fundamental de Android que representa una única pantalla con una interfaz de usuario. Es la unidad básica de interacción entre el usuario y la aplicación.



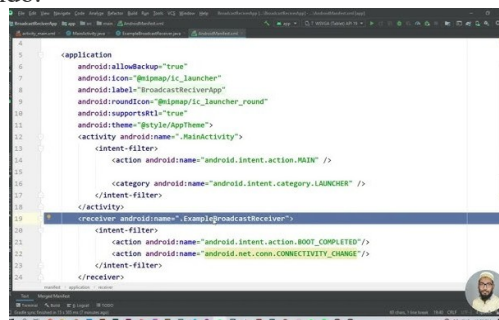
13. **Fragments**: Comportamiento o porción de interfaz de usuario reutilizable dentro de una Activity. Permite segmentar la lógica de la interfaz para adaptarse mejor a tablets o pantallas grandes.



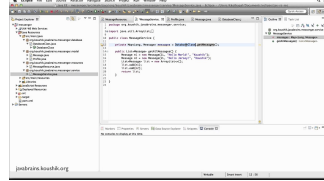
14. **Content Provider**: Componente que encapsula los datos y los pone a disposición de otras aplicaciones mediante una interfaz segura, permitiendo compartir información como contactos o archivos multimedia.



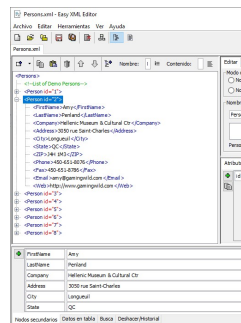
15. **Broadcast Receiver**: Componente que permite a la aplicación registrarse para recibir eventos del sistema o de otras aplicaciones (como avisos de carga de batería o recepción de SMS), incluso si la app no se está ejecutando.



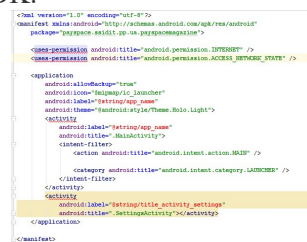
16. **Services:** Componente que puede realizar operaciones de larga ejecución en segundo plano sin proporcionar una interfaz de usuario (por ejemplo, reproducir música o descargar datos).



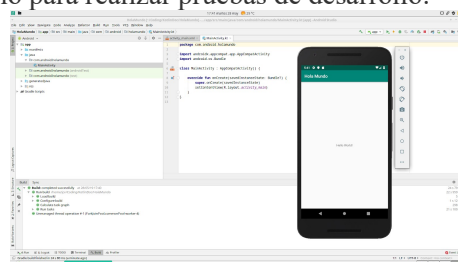
17. **XML (Extensible Markup Language):** Lenguaje de marcado utilizado en Android para estructurar datos y describir de forma declarativa la jerarquía de las interfaces de usuario y las configuraciones del sistema.



18. **Android Manifest:** Archivo binario (AndroidManifest.xml) obligatorio en toda app que describe información esencial al sistema operativo, incluyendo el nombre del paquete, componentes, permisos y versión mínima de SDK.



19. **Emulador:** Programa que permite simular las funciones de un dispositivo Android físico en una computadora de escritorio para realizar pruebas de desarrollo.



20.**Dispositivo**: El hardware físico real (teléfono, tablet, reloj inteligente) basado en la arquitectura Android donde se ejecuta la aplicación final.



21.**Android**: Sistema operativo basado en el kernel de Linux y otros softwares de código abierto, diseñado para dispositivos móviles con pantalla táctil.



22.**OS (Operating System)**: Sistema Operativo. Software que actúa de interfaz entre los dispositivos de hardware y el usuario, gestionando los recursos del sistema y la ejecución de programas.



23.**Android Studio**: El entorno de desarrollo integrado (IDE) oficial para la plataforma Android, basado en el software IntelliJ IDEA de JetBrains.



24. **APK** (Android Package Kit): Formato de archivo comprimido que Android utiliza para distribuir e instalar aplicaciones móviles y middleware.



25. **Bug**: Error, fallo o defecto en un programa de software que hace que este funcione de forma inesperada o se cierre abruptamente.



26. **Swap**: Técnica de gestión de memoria que consiste en utilizar una parte del almacenamiento (disco duro/memoria flash) como una extensión de la memoria RAM cuando esta se encuentra saturada.

```
1 import java.util.Scanner;
2 public class Example {
3     private static Scanner sc;
4     public static void main(String[] args)
5     {
6         int a, b, temp;
7         sc = new Scanner(System.in);
8
9         System.out.print(" Please Enter the First : ");
10        a = sc.nextInt();
11
12        System.out.print(" Please Enter the Second : ");
13        b = sc.nextInt();
14
15        System.out.println("\n Before : a = " + a + " and b = " + b);
16
17        temp = a;
18        a = b;
19        b = temp;
20
21        System.out.println("\n After : a = " + a + " and b = " + b);
22    }
23 }
```

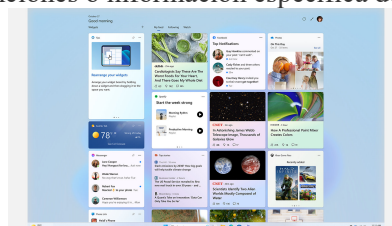
Output:

```
Please Enter the First : 10
Please Enter the Second : 25
Before : a = 10 and b = 25
After : a = 25 and b = 10
```

27. **UI (User Interface)**: Interfaz de Usuario. Espacio donde se producen las interacciones entre seres humanos y máquinas, enfocándose en la estética y la facilidad de uso de los elementos visuales.



28. **Widget**: Microaplicación o elemento visual de control que se integra en el escritorio del dispositivo para ofrecer acceso rápido a funciones o información específica de una app.



2. ¿Cuál es la relevancia de un ciclo de vida de un programa en Android?

La relevancia del ciclo de vida (Lifecycle) de un programa en Android radica en la gestión eficiente de recursos limitados (memoria RAM y batería) y en garantizar una experiencia de usuario fluida ante interrupciones constantes.

A diferencia de un programa de PC, donde una aplicación suele estar abierta hasta que el usuario la cierra, en Android el Sistema Operativo (SO) tiene el control total sobre la vida de los procesos.

Puntos clave de su importancia:

- **Optimización de Memoria RAM:** Android es un sistema multitarea. Si el usuario abre muchas apps, el sistema necesita liberar memoria. El ciclo de vida permite que una app "pause" o "detenga" sus procesos pesados cuando no está en pantalla, evitando que el teléfono se ralentice o se congele.
- **Conservación de la Batería:** Mediante los estados del ciclo de vida, la app puede detener servicios costosos (como el GPS, animaciones o sensores) en el momento en que el usuario deja de ver la pantalla, lo que ahorra energía significativamente.
- **Gestión de Interrupciones:** El ciclo de vida define cómo reacciona la app ante eventos externos, como recibir una llamada telefónica, la activación de una alarma o el cambio de orientación de la pantalla (de vertical a horizontal). Sin un ciclo de vida bien gestionado, la app podría perder los datos que el usuario estaba escribiendo o cerrarse inesperadamente.
- **Persistencia de Datos:** Permite al desarrollador decidir en qué momento exacto guardar información importante (por ejemplo, en el método `onPause()` o `onStop()`) para que, si el sistema destruye la app por falta de memoria, el usuario pueda retomar su actividad justo donde la dejó.

3. ¿Cuál es la mejor manera de realizar una ejecución de una Aplicación Móvil?

La relevancia del ciclo de vida en Android es fundamental porque, a diferencia de una computadora, un celular tiene recursos de memoria y batería muy limitados que deben cuidarse al máximo. Este ciclo permite que el sistema operativo gestione automáticamente cómo se comporta la aplicación cuando el usuario recibe una llamada, cambia de app o apaga la pantalla, asegurando que el teléfono no se ponga lento ni se consuma la batería innecesariamente. Para un programador, entender esto es la única forma de garantizar que el usuario no pierda sus datos si la aplicación se pausa o se cierra repentinamente por falta de memoria, ya que permite guardar el progreso justo antes de que la app deje de estar visible.

Respecto a la mejor manera de ejecutar una aplicación, lo más recomendable es usar un enfoque doble según el avance de tu proyecto. Durante la fase de escritura y diseño, lo más cómodo es el emulador de Android Studio porque te permite ver cambios rápidos en diferentes tamaños de pantalla sin conectar cables. Sin embargo, para la prueba definitiva de rendimiento, la mejor manera siempre será usar un dispositivo físico real, ya que es el único entorno donde puedes comprobar la sensibilidad del táctil, el uso real del GPS y cómo afecta el calor del procesador al funcionamiento de tu código antes de entregarlo.

En cuanto a la estructura del proyecto, el archivo más importante es el `AndroidManifest.xml` porque funciona como el mapa maestro y la identificación oficial de la aplicación ante el sistema operativo. Este archivo es vital porque es donde se declaran formalmente los permisos de la app, como el acceso a internet o a la cámara, y se define qué pantalla debe aparecer primero cuando el usuario toca el icono en su menú. Sin este manifiesto, el resto del código no serviría de nada, ya que el teléfono no tendría las instrucciones necesarias para saber qué componentes instalar o cómo debe arrancar la aplicación.

4. ¿Cuál es la estructura de un Proyecto en Android más importante? ¿Por qué?

La estructura de un proyecto en Android más importante es el archivo `AndroidManifest.xml`, el cual se encuentra ubicado en la raíz del conjunto de archivos de la aplicación y sirve como el documento de identidad principal que el sistema operativo consulta antes de permitir cualquier interacción con el software.

Este archivo es el más relevante porque funciona como el plano maestro y la partida de nacimiento de todo el proyecto ante el hardware del teléfono. Su importancia radica en que es el único lugar donde se declaran formalmente los permisos de seguridad, como el acceso a la cámara o al internet, y se define qué pantalla específica debe arrancar primero cuando el usuario toca el icono en su menú. Sin el Manifest, el resto del código y los diseños no tendrían ninguna instrucción válida para que el sistema operativo sepa cómo instalar o ejecutar la aplicación correctamente.

TEMA: INVESTIGACIÓN 1

Qué aprendí, Qué recordé, Qué reafirmé	Resumen
¿Qué aprendí? Aprendí la estructura interna de un proyecto en Android, identificando carpetas clave como Layout para el diseño visual y Values para recursos reutilizables. También comprendí el rol técnico de componentes avanzados como el Broadcast Receiver, que permite a la app reaccionar a eventos del sistema, y el Content Provider, que facilita el intercambio seguro de datos entre aplicaciones. ¿Qué recordé? Recordé las diferencias fundamentales entre aplicaciones nativas, híbridas y web, y cómo cada una impacta en el rendimiento y la experiencia del usuario. También refresqué el concepto de IDE y la importancia de contar con un entorno integrado como Android Studio para centralizar herramientas de diseño, programación y depuración en un solo lugar. ¿Qué reafirmé? Reafirmé que el AndroidManifest.xml es el archivo más crítico de cualquier proyecto, ya que sin sus declaraciones de permisos y componentes, el sistema operativo no puede ejecutar la app. Además, confirmé que el ciclo de vida es la columna vertebral del desarrollo móvil, pues garantiza que la aplicación sea eficiente en el uso de batería y memoria RAM.	En esta unidad de Herramientas de Programación IV, exploramos a fondo cómo se construye una aplicación de Android desde cero, entendiendo que no es solo escribir código, sino gestionar un ecosistema completo. Vimos la diferencia entre hacer una app nativa, que aprovecha todo el poder del celular, y una híbrida o web, que son más flexibles pero limitadas. Entramos en la estructura del proyecto en Android Studio, donde aprendí que carpetas como Layout y Values son las que dan vida a la interfaz, mientras que componentes como el Activity y los Fragments controlan lo que el usuario ve y hace. También profundizamos en herramientas técnicas como el SDK Manager para mantener todo actualizado y el uso de XML para que el diseño sea ordenado y profesional.

Conclusiones

- La conclusión más importante que me llevo es que el éxito de una aplicación móvil depende totalmente de qué tan bien manejemos el ciclo de vida. Como los celulares no tienen memoria infinita, si no programamos pensando en ahorrar batería y RAM, la app simplemente no va a servir en el mundo real. También me quedó claro que el AndroidManifest.xml es el corazón de todo; si ese archivo está mal, nada funciona porque el sistema operativo no sabría ni qué permisos tiene la app. Por último, aprendí que la mejor forma de trabajar es probando constantemente en el emulador para avanzar rápido, pero nunca saltarse la prueba en un celular físico para estar seguro de que la experiencia del usuario sea fluida y sin errores antes de generar el APK final.